DOUBLE BALL GOMOKU — AI DESIGN AND IMPLEMENTATION REPORT

---

1. Introduction

This project presents **Double Ball Gomoku**, a 15×15 grid-based strategy game inspired by classical Gomoku but extended with two separate AI systems: a **Defender AI** and an **Attacker AI**. The game is implemented in Python using Pygame for graphical interaction and NumPy for board management. The Defender AI prioritizes **threat blocking and loss prevention**, making it reactive, while the Attacker AI emphasizes **pattern extension and offensive expansion**, making it proactive.

Both AIs use **heuristic-driven board pattern detection** rather than deep Minimax search, enabling real-time decision-making even within a large state-space of roughly 225 playable positions. This report thoroughly documents system design, AI heuristics, pattern logic, complexity, and implementation choices behind both versions of Double Ball Gomoku.

---

2. Theoretical Background

Gomoku is a **deterministic, perfect-information, zero-sum board game**. The objective is simple: form a line of five stones horizontally, vertically, or diagonally. However, the search space grows extremely fast. On a 15×15 board, after the opening move:

- Remaining possible moves = 224
- Branching factor ≈ 100–200 mid-game
- Traditional Minimax search is **computationally infeasible** without pruning or heuristics.

Thus, Double Ball Gomoku AI depends on:

2.1 Pattern-Based Heuristic Search

Instead of exploring future game-tree states, the AI directly evaluates the current board using sequence detection:

- Count how many identical consecutive stones exist on each line.
- Identify patterns that indicate immediate threats or opportunities.
- Decide the best move based on local tactical value.

This approach is inspired by:

- Local search
- Threat-space search
- Pattern scoring methods used in competitive Gomoku programs

2.2 Real-Time Decision Requirement

Because the game is interactive (human vs AI), all computations must occur **within milliseconds**.
Heuristic detection provides:

- Fast evaluation
- Sufficient tactical strength
- Clear interpretability of moves

This justifies the choice of simple linear scans and urgency-based decision models instead of deep Minimax.

---

3. Game Overview and Rules

3.1 Board Setup

- 15×15 grid
- Represented as a 2D NumPy array
- `0 = empty`, `1 = attacker stone`, `2 = defender stone`
- Stones are represented visually using custom images (`football.png`, `black.png`)

3.2 Win Condition

A player wins by forming **five consecutive identical stones** in:

- Horizontal line
- Vertical line
- Diagonal (↘)
- Anti-diagonal (↙)

3.3 Versions Implemented

There are **two complete versions** of Double Ball Gomoku:

Version A — Defender AI (main game)

- Player = Attacker
- AI = Defender
- Files: `my_ai.py`, `ai_move.py`

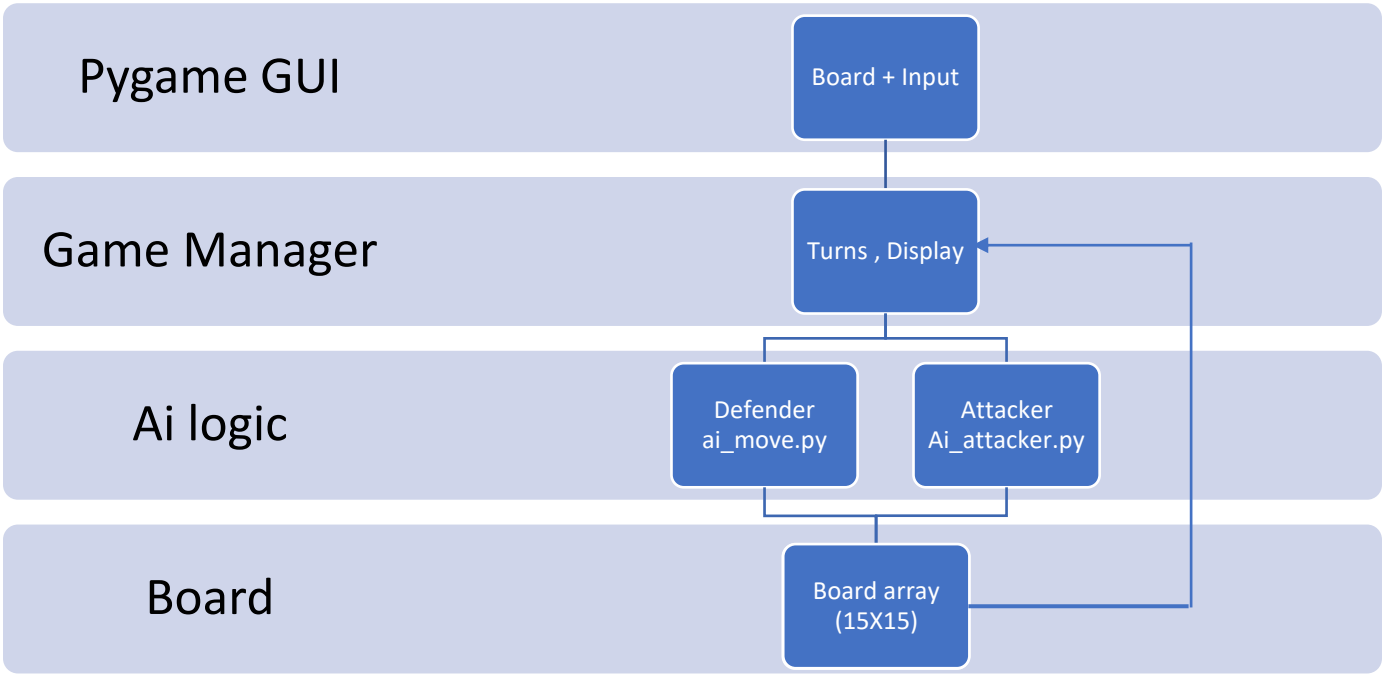
Version B — Attacker AI (opposite mode)

- AI = Attacker
- Player = Defender
- Files: `my_ai_attacker.py`, `ai_attacker.py`

Each version behaves differently and uses a separate heuristic system.

---

4. System Architecture



---

5. Win Detection Algorithm (game_arr.py)

Win detection scans the board using **four directional checks**, each looking for sequences of length 5.

5.1 Horizontal Scan

For each row:

- Slide across all sets of 5 consecutive cells
- Check if they contain identical non-zero values

5.2 Vertical Scan

Same logic, but scanning columns.

5.3 Main Diagonal (↘) Scan

Check diagonals starting from top-left.

5.4 Anti-Diagonal (↙) Scan

Check diagonals starting from top-right.

5.5 Pseudocode

```
for each direction in {horizontal, vertical, diag_down, diag_up}:
    slide window of length 5:
        if all 5 values equal and not zero:
            return WIN
return NO WIN
```

6. Defender AI (ai_move.py)

Purpose:

To **block threats**, **prevent immediate player wins**, and opportunistically form defender sequences.

6.1 Strategy Overview

The Defender AI follows a strict priority order:

1. **Block or complete length-4 sequences**
2. **Block or complete length-3 sequences**
3. **Block or complete length-2 sequences**
4. If no patterns found → **random empty cell**

This makes the AI **reactive** and defensive.

6.2 Step-by-Step Logic

Step 1 — Compute Sequence Scores (check function)

The `check(arr)` function creates a **score matrix (15×15)** by detecting all existing horizontal, vertical, and diagonal consecutive stone chains.

It does NOT choose moves—only evaluates the board.

Step 2 — Pattern-Based Move Selector (aimove)

For each length in [4, 3, 2]:

Evaluate all 5-cell windows:

- If window contains **length identical stones of player (1)** → AI places `2` to block
- If window contains **length identical stones of AI (2)** → AI places `1` to complete

This ensures:

Highest priority: Stop opponent from winning

Second priority: Extend AI's own sequences

Pseudocode

```
for length in [4,3,2]:
    scan all horizontal 5-cells:
        if segment has length stones for defender AND empty cell:
            place defender stone there
            return
        if segment has length stones for attacker AND empty cell:
            block attacker
```

```
            return
    repeat scanning vertical, diag(↘), diag(↙)
If no move found:
    choose random empty cell
```

---

7. Attacker AI (ai_attacker.py)

Purpose:

To aggressively build longer sequences and put the opponent under pressure.

7.1 Strategy Overview

The Attacker AI checks sequence lengths in **reverse order** compared to Defender AI:

1. **Extend 4-in-a-row to win**
2. **Extend 3-in-a-row to threaten**
3. **Extend 2-in-a-row**
4. **Extend even 1 stone**
5. If no line extension possible → **place near stones**
6. Final fallback → **random empty cell**

7.2 Offensive Behavior

Unlike the Defensive AI, it **prioritizes growth** instead of blocking.

Pseudocode

```
for length in [4,3,2,1]:
    for each 5-segment in all directions:
        if segment contains 'length' stones of same color:
            try placing adjacent to extend the chain
            return
If no extension possible:
    pick empty cell adjacent to any stone
If none found:
    choose random empty cell
```

---

8. Comparison Between Attacker and Defender AI

Feature	*Defender AI*	*Attacker AI*
Primary Goal	Block, prevent loss	Extend, create threats
Priority Rules	4 → 3 → 2	4 → 3 → 2 → 1
Behavior Style	Reactive, risk-averse	Proactive, aggressive
Strengths	Hard to defeat, good at blocking	Creates pressure, can force wins
Weaknesses	Limited offensive intelligence	Defense is weaker
Use Case	Player attacks	AI attacks

Both AI systems complement each other and demonstrate different strategic ideologies.

---

9. Complexity Analysis

9.1 Board Evaluation

All scans operate in:

$O(N^2 \times K)$

Where:

- N = 15
- K = 5 (sequence window)

For practical purposes → **very fast**.

9.2 AI Move Computation

Both AIs scan ~225 cells × 5-cell windows.

Worst-case:

$O(225 \times 5) \approx O(1125)$ operations

≈ **1 millisecond** on modern hardware.

9.3 Space Complexity

Board: 15×15
Score array: 15×15

Total: $O(N^2)$

---

10. Design Justification

10.1 Why No Minimax?

A realistic Minimax for Gomoku requires depth 3–5 for meaningful play:

- Branching: 80–200 moves per turn
- Depth 3 → $80^3 = 512{,}000$ nodes
- Depth 4 → > 40 million nodes

This is too slow for a casual Pygame project without optimization.

10.2 Why Heuristics?

- Fast
- Easy to understand
- Matches human tactical instincts
- Allows two completely different AI personalities

10.3 Why Two Versions?

To demonstrate:

- How different heuristics change AI personality
- The contrast between reactive and proactive Intelligent Agents
- Flexibility and depth of your game engine

This dual-system design significantly strengthens your project.

---

11. Conclusion

Double Ball Gomoku successfully implements two full AI subsystems demonstrating opposing philosophies of game strategy:

- **Defender AI** — defensive, threat-based, reactive
- **Attacker AI** — offensive, sequence-extending, proactive

The use of pattern detection and heuristic-driven decision-making ensures real-time responsiveness without requiring deep game-tree exploration. The architecture is modular, readable, and easily extendable into more advanced algorithms (Minimax, Monte Carlo Tree Search, etc.).

This project shows strong understanding of:

- Game state representation
- Heuristic evaluation
- Pattern scanning
- Real-time AI decision-making
- Turn-based game architecture using Pygame

It forms an excellent foundation for further exploration in game AI and adversarial search techniques.
